



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorios de computación salas A y B

Profesor(a):

Miguel Ramírez

Asignatura:

Modelos de Programación Orientada a Objetos

Grupo:

01

No. de práctica(s):

#04 Variables y arreglos

Integrante(s):

Quintana Fuentes Melanie Marifer

No. de lista o brigada:

#14

Semestre:

2027-1

Fecha de entrega:

18 de septiembre de 2026

Observaciones:

CALIFICACIÓN: _____

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	52/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Guía práctica de estudio 04: Variables y arreglos



Elaborado por:

M.C. M. Angélica Nakayama C.
Ing. Jorge A. Solano Gálvez

Autorizado por:

M.C. Alejandro Velázquez Mena

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	53/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Guía práctica de estudio 04: Variables y arreglos

Objetivo:

Utilizar variables y arreglos con tipos de datos primitivos y con bibliotecas propias del lenguaje.

Actividades:

- Crear variables con diferentes tipos de datos.
- Crear arreglos con diferentes tipos de datos.

Introducción

Los **tipos de datos** hacen referencia al tipo de información que se trabaja, donde la unidad mínima de almacenamiento es el dato, también se puede considerar como el rango de valores que puede tomar una variable durante la ejecución del programa. Un tipo de datos define un conjunto de valores y las operaciones sobre estos valores.

Casi todos los lenguajes de programación explícitamente incluyen la notación del tipo de datos, aunque lenguajes diferentes pueden usar terminologías diferentes. La mayor parte de los lenguajes de programación permiten al programador definir tipos de datos adicionales, normalmente combinando múltiples elementos de otros tipos y definiendo las operaciones del nuevo tipo de dato.

Los **valores** que pueden adquirir los **tipos de datos** se manipulan durante la ejecución de un programa a través de **variables** o **arreglos**.

NOTA: En esta guía se tomará como caso de estudio el lenguaje de programación JAVA, sin embargo, queda a criterio del profesor el uso de éste u otro lenguaje orientado a objetos.

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	54/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Variable

Una **variable** es un **nombre** que contiene un valor que **puede cambiar** a lo largo del programa. De acuerdo con el tipo de información que contienen, en Java hay dos tipos principales de variables:

1. Variables de tipos primitivos.
2. Variables de referencia.

Desde el punto de vista del papel o misión en el programa, las variables pueden ser:

- Variables **miembro** de una clase: Se definen en una clase, fuera de cualquier método; pueden ser tipos primitivos o referencias.
- Variables **locales**: Se definen dentro de un método o, de forma más general dentro de cualquier bloque entre llaves { }. Se crean en el interior del bloque y se destruyen al finalizar dicho bloque.

Una variable se define especificando el **tipo de dato** y el **nombre** de dicha variable. Las variables pueden ser tanto de tipos primitivos como referencias a objetos de alguna clase perteneciente al API de Java o creada por el usuario.

tipoDeDato *nombreVariable*;

Ejemplos:

int *miVariable*;

float *area*;

char *letra*;

String *cadena*;

MiClase *prueba*;

Si no se especifica un valor en su declaración, las variables miembro primitivas se inicializan a **cero** (salvo boolean y char, que se inicializan a false y '\0', respectivamente). Así mismo,

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	55/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

las variables miembro de tipo referencia son inicializadas por defecto a un valor especial: **null**.

Es importante distinguir entre la referencia a un objeto y el objeto mismo. Una referencia es una variable que indica dónde está guardado un objeto en la memoria (a diferencia de C/C++, Java no permite acceder al valor de la dirección, pues en este lenguaje se han eliminado los apuntadores). Al declarar una referencia todavía no se encuentra “apuntando” a ningún objeto en particular (salvo que se cree explícitamente un nuevo objeto en la declaración), y por eso se le asigna el valor **null**.

Si se desea que esta referencia apunte a un nuevo objeto es necesario crear el objeto utilizando el operador **new**. Este operador reserva espacio en la memoria para el objeto (variables y funciones).

También es posible igualar la referencia declarada a otra referencia a un objeto existente previamente.

Ejemplo:

```
MyClass unaRef;
unaRef = new MyClass();
MyClass segundaRef = unaRef;
```

Un tipo particular de referencias son los **arrays** o **arreglos**, sean éstos de variables primitivas (por ejemplo, de enteros) o de objetos. En la declaración de una referencia de tipo **array** hay que incluir los **corchetes** [].

Ejemplo:

```
int [ ] vector;
vector = new int [10];
MyClass [ ] lista=new MyClass [5];
```

En Java todas las variables deben estar incluidas en alguna clase. En general las variables declaradas dentro de llaves { }, es decir, dentro de un bloque, son visibles y existen dentro de estas llaves. Por ejemplo las variables declaradas al principio de un método existen mientras se ejecute el método; las variables declaradas dentro de un bloque if no serán válidas al finalizar las sentencias correspondientes a dicho if y las variables miembro de una

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	56/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

clase (es decir declaradas entre las llaves { } de la clase pero fuera de cualquier método) son válidas mientras existe el objeto de la clase.

Constante

Una **constante** es una variable cuyo valor **no puede ser modificado**. Para definir una constante en Java se utiliza la palabra reservada **final**, antes de la declaración del tipo de dato, de la siguiente manera:

final tipoDato nombreDeConstante = valor;

Ejemplo:

final double PI = 3.1416;

Arreglo

Un **arreglo** es un objeto en el que se puede almacenar un conjunto de datos de un **mismo tipo**. Cada uno de los elementos del arreglo tiene asignado un **índice** numérico según su posición, siendo 0 el índice del primer elemento. Se declara de la siguiente manera:

tipoDeDato [] nombreVariable; o *tipoDeDato* nombreVariable[];

Como se puede apreciar, los corchetes pueden estar situados delante del nombre de la variable o detrás. Ejemplos:

int [] k; *String* [] p; *char* datos[];

Los arreglos pueden declararse en los mismos lugares que las variables estándar. Para asignar un tamaño al arreglo se utiliza la expresión:

variableArreglo = **new** *tipoDeDato*[tamaño];

También se puede asignar el tamaño al arreglo en la misma línea de declaración de la variable.

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	57/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

```
int [ ] k = new int[5];
```

Cuando un arreglo se dimensiona, todos sus elementos son inicializados explícitamente al **valor por defecto** del tipo correspondiente.

Para declarar, dimensionar e inicializar un arreglo en una misma sentencia se indican los valores del arreglo entre llaves y separados por comas. Ejemplo:

```
int [ ] nums = {10, 20, 30, 40};
```

El acceso a los elementos de un arreglo se realiza utilizando la expresión:

variableArreglo[índice]

Donde índice representa la posición a la que se quiere tener acceso y cuyo valor debe estar entre **0 y tamaño - 1**

Todos los objetos arreglo exponen un atributo publico llamado **length** que permite conocer el tamaño al que ha sido dimensionado un arreglo.

Ejemplo:

```
int [ ] nums = new int[10];
for(int i=0; i < nums.length; i++)
    nums[i] = i * 2;
```

Los arreglos al igual que las variables, se pueden usar como argumentos, así como también pueden ser devueltos por un método o función.

En Java se puede utilizar una variante del ciclo for llamado **for-each**, para facilitar el recorrido de arreglos y colecciones, recuperando su contenido y eliminando la necesidad de usar una variable de control que sirva de índice. Su sintaxis es:

```
for(tipoDato variable: variableArreglo){
    //instrucciones
}
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	58/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Ejemplo:

```
int [] nums = { 4, 6, 30, 15 };
for(int n : nums){
    System.out.println(n);
}
```

En este caso, sin acceder de forma explícita a las posiciones del arreglo, cada una de estas es copiada automáticamente a la variable auxiliar **n** al principio de cada iteración.

Los arreglos en Java también pueden tener más de una dimensión, al igual que en C/C++ para declarar un arreglo bidimensional se tendrían que usar dos pares de corchetes y en general para cada dimensión se usa un nuevo par de corchetes.

Ejemplo:

```
int [ ] [ ] matriz;
```

Argumentos por línea de comandos

Es posible suministrar parámetros al método **main** a través de la línea de comandos. Para ello, los valores a pasar deberán especificarse a continuación del nombre de la clase separados por un espacio:

```
>> java NombreClase arg1 arg2 arg3
```

Los datos llegarán al método **main** en forma de un arreglo de cadenas de caracteres.

Ejemplo:

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	59/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

```

public class Ejemplo {

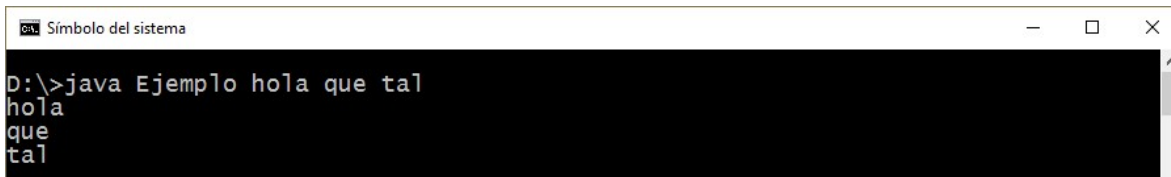
    public static void main(String[] args) {
        System.out.println(args[0]);
        System.out.println(args[1]);
        System.out.println(args[2]);
    }

}

```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	60/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Se ejecuta utilizando la siguiente expresión en la línea de comandos:



```
D:\>java Ejemplo hola que tal
hola
que
tal
```

API de JAVA

La API Java (**A**pplication **P**rogramming **I**nterface) es una interfaz de programación de aplicaciones provista por los creadores del lenguaje, que da a los programadores los medios para desarrollar aplicaciones Java.

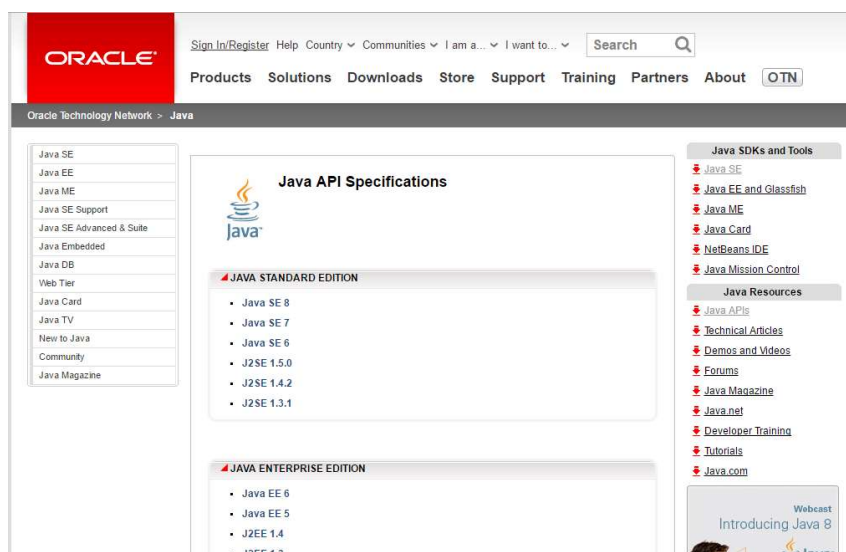
Como el lenguaje Java es un lenguaje orientado a objetos, la **API** de Java provee de un conjunto de clases utilitarias para efectuar toda clase de tareas necesarias dentro de un programa. La **API** Java está organizada en paquetes lógicos, donde cada paquete contiene un conjunto de clases relacionadas semánticamente.

La información completa del **API** de java se denomina **especificación** y sirve para conocer cualquier aspecto sobre las clases que contiene la **API**. Esta especificación es de crucial importancia para los programadores ya que en ella se pueden consultar los detalles de alguna clase que se quiera utilizar y ya no sería necesario memorizar toda la información relacionada.

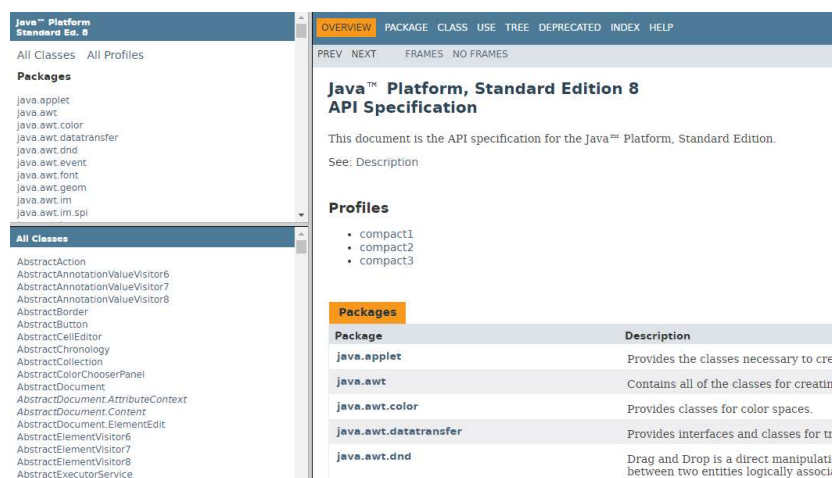
La especificación del **API** de java se encuentra en el sitio de Oracle: <http://www.oracle.com/technetwork/java/api-141528.html>.

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	61/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

En este sitio se puede consultar la especificación de las últimas versiones (ya que en cada versión se agregan o modifican algunas clases).



Una vez seleccionada la versión, se puede consultar el detalle de todas las clases que integran el API.



	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	62/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Manejo de cadenas

En Java las cadenas de caracteres no son un tipo de datos primitivo, sino que son objetos pertenecientes a la clase **String**.

La clase **String** proporciona una amplia variedad de métodos que permiten realizar las operaciones de manipulación y tratamiento de cadenas de caracteres habituales en un programa.

Para crear un objeto **String** podemos seguir el procedimiento general de creación de objetos en Java, utilizando el operador **new**. Ejemplo:

```
String s = new String("Texto de prueba");
```

Sin embargo, dada la amplia utilización de estos objetos en un programa, Java permite crear y asignar un objeto String a una variable de la misma forma que se hace con los tipos de datos primitivos. Entonces el ejemplo anterior es equivalente a:

```
String s = "Texto de prueba";
```

Una vez creado el objeto y asignada la referencia al mismo a una variable, puede utilizarse para acceder a los métodos definidos en la clase String (se pueden revisar en la documentación del API con java.lang.String). Los más usados son: length, equals, charAt, substring, indexOf, replace, toUpperCase, toLowerCase, Split, entre otros.

Ejemplo:

```
s.length();           //Devuelve el tamaño de la cadena
```

```
s.toUpperCase();      //Devuelve la cadena en mayúsculas
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	63/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Las variables de tipo String se pueden usar en una expresión que use el operador + para concatenar cadenas. Ejemplo:

```
String s = "Hola";
```

```
String t = s + " qué tal";
```

En Java las cadenas de caracteres son objetos **inmutables**, esto significa que una vez que el objeto se ha creado, no puede ser modificado.

Cuando escribimos una instrucción como la del ejemplo anterior, es fácil intuir que la variable *s* pasa a apuntar al objeto de texto *"Hola que tal"*. Aunque a raíz de la operación de concatenación pueda parecer que el objeto *"Hola"* apuntado por la variable *s* ha sido modificado en realidad no sucede esto, sino que al concatenarse *"Hola"* con *" que tal"* se está creando un nuevo objeto de texto *"Hola que tal"* que pasa a ser referenciado por la variable *s*. El objeto *"Hola"* deja de ser referenciado por dicha variable.

Java provee soporte especial para la concatenación de cadenas con las clases **StringBuilder** y **StringBuffer**. Un objeto **StringBuilder** es una secuencia de caracteres **mutable**, su contenido y capacidad puede cambiar en cualquier momento. Además, a diferencia de los **Strings**, los **builders** cuentan con una capacidad (capacity), la cantidad de espacios de caracteres asignados. Ésta es siempre mayor o igual que la longitud (length) y se expande automáticamente para acomodarse a más caracteres.

Los métodos principales de la clase **StringBuilder** son **append** e **insert**. Cada uno convierte un dato en **String** y concatena o inserta los caracteres de dicho String al **StringBuilder**. El método **append** agrega los caracteres al final mientras que **insert** los agrega en un punto específico.

Para hacer la misma concatenación que el ejemplo con String, quedaría:

```
StringBuilder sb = new StringBuilder("Hola");
```

```
sb.append(" que tal");
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	64/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Wrappers

Los **wrappers** o clases envoltorio son clases diseñadas para ser un complemento de los tipos primitivos. En efecto, los tipos primitivos son los únicos elementos de Java que no son objetos. Esto tiene algunas ventajas desde el punto de vista de la eficiencia, pero algunos inconvenientes desde el punto de vista de la funcionalidad.

Por ejemplo, los tipos primitivos siempre se pasan como argumento a los métodos por valor, mientras que los objetos se pasan por referencia. No hay forma de modificar en un método un argumento de tipo primitivo y que esa modificación se trasmita al entorno que hizo la llamada.

Una forma de conseguir esto es utilizar un **wrapper**, esto es un objeto cuya variable miembro es el tipo primitivo que se quiere modificar. Las clases **wrapper** también proporcionan métodos para realizar otras tareas con los tipos primitivos, tales como conversión con cadenas de caracteres en uno y otro sentido.

Existe una clase **wrapper** para cada uno de los tipos primitivos, las clases son: Byte, Short, Character, Integer, Long, Float, Double y Boolean (obsérvese que los nombres empiezan por mayúscula, siguiendo la nomenclatura típica de Java). Todas estas clases se encuentran en *java.lang*.

Todas las clases **wrapper** permiten crear un objeto de la clase a partir de tipo básico.

```
int k = 23;
```

```
Integer num = new Integer(k);
```

A excepción de Character, las clases **wrapper** también permiten crear objetos partiendo de la representación como cadena del dato.

```
String s = "4.65";
```

```
Float ft = new Float(s);
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	65/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Para recuperar el valor a partir del objeto, las ocho clases **wrapper** proporcionan un método con el formato *xxxValue()* que devuelve el dato encapsulado en el objeto donde xxx representa el nombre del tipo en el que se quiere obtener el dato.

```
float dato = ft.floatValue( );
```

```
int n=num.intValue( );
```

Las clases numéricas proporcionan un método estático *parseXxx(String)* que permite convertir la representación en forma de cadena de un número en el correspondiente tipo numérico donde xxx es el nombre del tipo al que se va a convertir la cadena de caracteres.

```
String s1 = "25, s2="89.2";
```

```
int n = Integer.parseInt(s1);
```

```
double d = Double.parseDouble(s2);
```

Auto boxing y unboxing

El **autoboxing** consiste en la encapsulación automática de un dato básico en un objeto wrapper mediante la utilización del operador de asignación.

Por ejemplo:

```
int p = 5;
Integer n = new Integer(p);
```

Equivale a:

```
int p = 5;
Integer n = p;
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	66/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Es decir, la creación del objeto **wrapper** se produce implícitamente al asignar el dato primitivo a la variable objeto. De la misma forma, para obtener el dato básico a partir del objeto **wrapper** no será necesario recurrir al método *xxxValue()*, esto se realizará implícitamente al utilizar la variable objeto en una expresión. A esto se le conoce como **autounboxing**. Para el ejemplo anterior:

```
int a = n;
```

	Manual de prácticas del Laboratorio de Modelos de programación orientada a objetos	Código:	MADO-21
		Versión:	02
		Página	67/166
		Sección ISO	8.3
		Fecha de emisión	14 de junio de 2018
Facultad de Ingeniería		Área/Departamento: Laboratorio de computación salas A y B	
La impresión de este documento es una copia no controlada			

Bibliografía

Martín, Antonio

Programador Certificado Java 2.

Segunda Edición.

México

Alfaomega Grupo Editor, 2008

Sierra Katy, Bates Bert

SCJP Sun Certified Programmer for Java 6 Study Guide

Mc Graw Hill

Dean John, Dean Raymond.

Introducción a la programación con Java

Primera Edición.

México

Mc Graw Hill, 2009

PRÁCTICA #4

Variables y arreglos

Enlace de HackerRank: <https://www.hackerrank.com/mpoo-2027-1-practica4>

1. Juste en la montaña rusa

Antes de iniciar operaciones, el sistema de mantenimiento de una montaña rusa revisa los valores de calibración de los vagones que forman el tren. Para autorizar su operación, estos valores deben encontrarse ordenados de forma ascendente desde el primer vagón hasta el último.

Durante una inspección, algunos vagones pueden encontrarse fuera de la posición que les corresponde. El personal de mantenimiento puede seleccionar un único tramo continuo y reorganizar solamente los valores contenidos dentro de ese tramo.

Actividad

Determina el tramo continuo más pequeño posible que, al ordenarse de forma ascendente, permita que toda la formación quede correctamente ordenada. El resultado deberá indicar el índice inicial, el índice final y la longitud del tramo. **Los índices comienzan en 0.**

Si el arreglo ya se encuentra ordenado, el método deberá regresar:

```
[-1, -1, 0]
```

Ejemplo

Para la formación:

```
1 2 4 7 10 11 7 12 6 7 16 18 19
```

El tramo mínimo que necesita reorganización se encuentra entre los índices 3 y 9. Por lo tanto, el resultado esperado es:

```
3 9 7
```

El método debe regresar el arreglo con estos tres valores.

Evidencias: Prueba tu código mediante HackerRank, agrega una captura del código fuente y otra donde hayan pasado todos los casos de prueba. También recuerda subir tus códigos a Github.

Como parte del reporte, incluye una explicación y/o diagrama de flujo que demuestre claramente el razonamiento utilizado para resolver el problema, sin esto, no será válido el ejercicio.

Código fuente:

```
18     public static int[] detectarZonaAjuste(int[] vagones) {
19
20         int n = vagones.length;
21
22         int inicio = 0;
23
24         while (inicio < n - 1 && vagones[inicio] <= vagones[inicio + 1]) {
25             inicio++;
26         }
27
28         if (inicio == n - 1) {
29             return new int[]{-1, -1, 0};
30         }
31
32         int fin = n - 1;
33
34         while (fin > 0 && vagones[fin - 1] <= vagones[fin]) {
35             fin--;
36         }
37
38         int minimo = vagones[inicio];
39         int maximo = vagones[fin];
40
41         for (int i = inicio; i <= fin; i++) {
42
43             if (vagones[i] < minimo) {
44                 minimo = vagones[i];
45             }
46
47             if (vagones[i] > maximo) {
48                 maximo = vagones[i];
49             }
50         }
51
52         while (inicio > 0 && vagones[inicio - 1] > minimo) {
53             inicio--;
54         }
55
56         while (fin < n - 1 && vagones[fin + 1] < maximo) {
57             fin++;
58         }
59
60         int longitud = fin - inicio + 1;
61
62         return new int[]{inicio, fin, longitud};
63     }
64
65
66 }
```

Ejecución:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

1 1 2 3 4 5 6 7

✓ Sample Test case 2

Your Output (stdout)

1 -1 -1 0

Expected Output

[Download](#)

1 -1 -1 0

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

✓ Test case 0

Compiler Message

Success

✓ Test case 1

✓ Test case 2

Input (stdin)

[Download](#)

1 1 2 3 4 5 6 7

✓ Test case 3

✓ Test case 4

Expected Output

[Download](#)

1 -1 -1 0

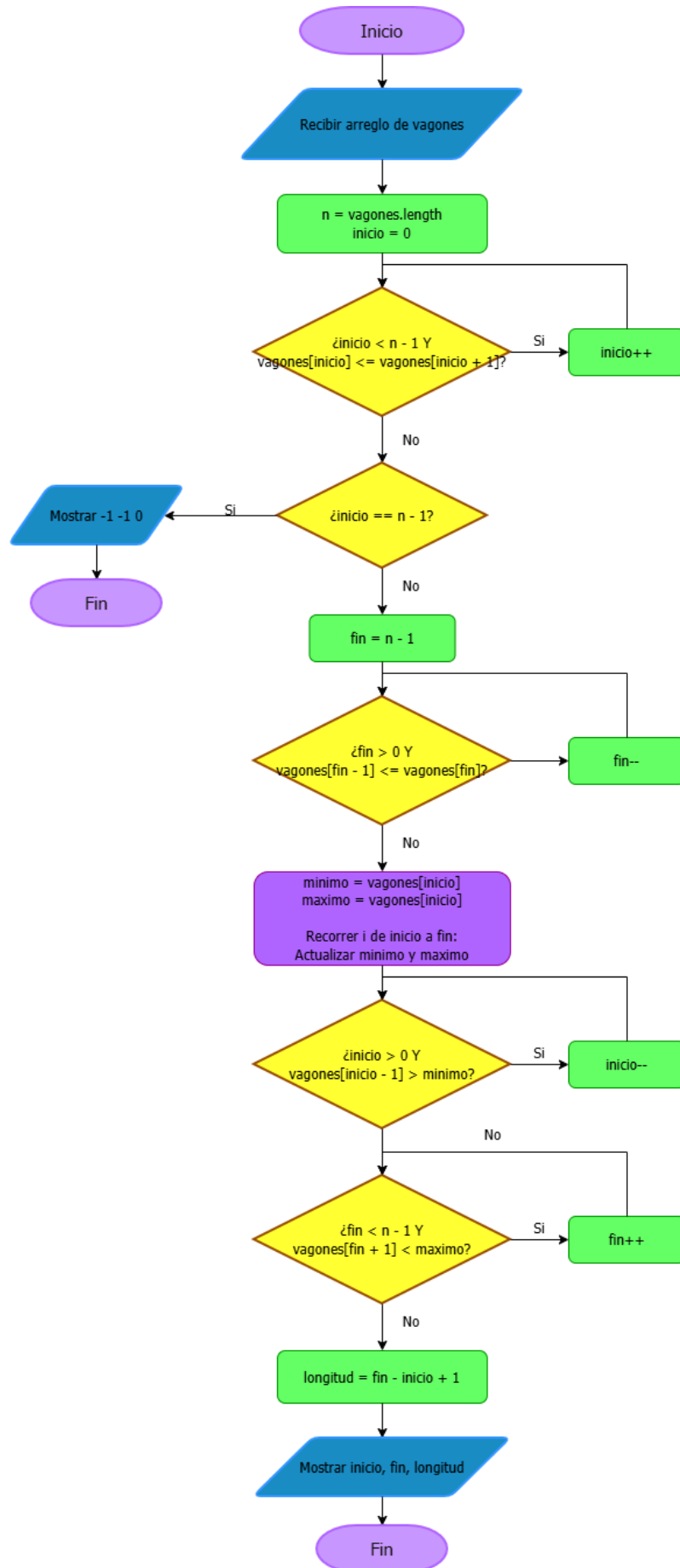
✓ Test case 5

✓ Test case 6

✓ Test case 7

✓ Test case 8

✓ Test case 9



Es un algoritmo de dos punteros que encuentra la zona mínima de ajuste de 3 fases:

1. Recorre la formación desde los extremos hacia el centro para omitir los vagones que están bien colocados, si el tren completo está ordenado, finaliza devolviendo $[-1, -1, 0]$.
2. Analiza la subcadena desordenada para encontrar los valores mínimos y máximos, con estos expande los límites hacia los lados si es que existen elementos fuera del tramo que afecten la posición final.
3. Calcula el tamaño de la zona con longitud = fin - inicio + 1 y retorna la ubicación exacta con la cantidad de vagones a reordenar.

2. La rueda del mensajero

En una antigua confederación, los gobernantes enviaban órdenes mediante mensajeros que recorrían grandes distancias entre ciudades. Para evitar que el contenido de una carta pudiera ser entendido si el mensajero era interceptado, los escribas utilizaban un dispositivo formado por dos ruedas concéntricas.

Actividad

La rueda exterior permanece fija y contiene el alfabeto de la A-Z. La rueda interior contiene las mismas 26 letras, pero en un orden secreto. Cada letra de la rueda exterior se asocia con la letra que ocupa la misma posición en la rueda interior.

Antes de comenzar el cifrado, la rueda interior deberá girarse hacia la derecha la cantidad de posiciones indicada por `posicionInicial`. Después de procesar cada `intervaloRotacion` letras, la rueda deberá girar una posición adicional hacia la derecha. Los espacios se conservan y no cuentan como letras procesadas.

Implementa el siguiente método. Debe regresar el mensaje cifrado:

```
public static String cifrarMensaje(  
    String mensaje,  
    String alfabetoInterior,  
    int posicionInicial,  
    int intervaloRotacion)
```

Ejemplo

Entrada	Resultado esperado
Mensaje: ATAQUE Alfabeto interior: QWERTYUIOPASDFGHJKLZXCVBNM Posición inicial: 0 intervalo de rotación: 4	QZQJZR

Mensaje: HOLA MUNDO Alfabeto interior: QWERTYUIOPASDFGHJKLZXCVBNM Posición inicial: 3 Intervalo de rotación: 5	TSOB PJPMA
--	------------

La entrada utilizada por HackerRank contiene cuatro líneas: mensaje, alfabetoInterior, posicionInicial e intervaloRotacion. El método deberá realizar únicamente el mecanismo de cifrado.

Evidencias: Prueba tu código mediante HackerRank, agrega una captura del código fuente y otra donde hayan pasado todos los casos de prueba. También recuerda subir tus códigos a Github.

Como parte del reporte, incluye una explicación y/o diagrama de flujo que demuestre claramente el razonamiento utilizado para resolver el problema, sin esto, no será válido el ejercicio.

Código fuente:

```

9  class Result {
21
22      public static String cifrarMensaje(String mensaje, String alfabetoInterior,
    int posicionInicial, int intervaloRotacion) {
23
24          StringBuilder resultado = new StringBuilder();
25          int contadorLetras = 0;
26          int rotacion = posicionInicial;
27
28          for (int i = 0; i < mensaje.length(); i++) {
29              char c = mensaje.charAt(i);
30
31              if (c == ' ') {
32                  resultado.append(' ');
33                  continue;
34              }
35
36              int posExterior = c - 'A';
37
38              int posInterior = (posExterior - rotacion) % 26;

```

```

39  ✓   if (posInterior < 0) {
40      |       posInterior += 26;
41      |   }
42
43      resultado.append(alfabetoInterior.charAt(posInterior));
44
45      contadorLetras++;
46
47  ✓   if (intervaloRotacion > 0 && contadorLetras % intervaloRotacion == 0) {
48      |       rotacion = (rotacion + 1) % 26;
49      |   }
50
51      }
52
53  ✓   return resultado.toString();
54
55      }

```

Ejecución:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

✓ Sample Test case 2

```

1  ATAQUE
2  QWERTYUIOPASDFGHJKLZXCVBNM
3  0
4  4

```

Your Output (stdout)

```

1  QZQJZR

```

Expected Output

[Download](#)

```

1  QZQJZR

```

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

✓ Test case 0

✓ Test case 1 

✓ Test case 2

✓ Test case 3 

✓ Test case 4

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

```
1  ATAQUE
2  QWERTYUIOPASDFGHJKLZXCVBNM
3  0
4  4
```

[Download](#)

Expected Output

```
1  QZQJZR
```

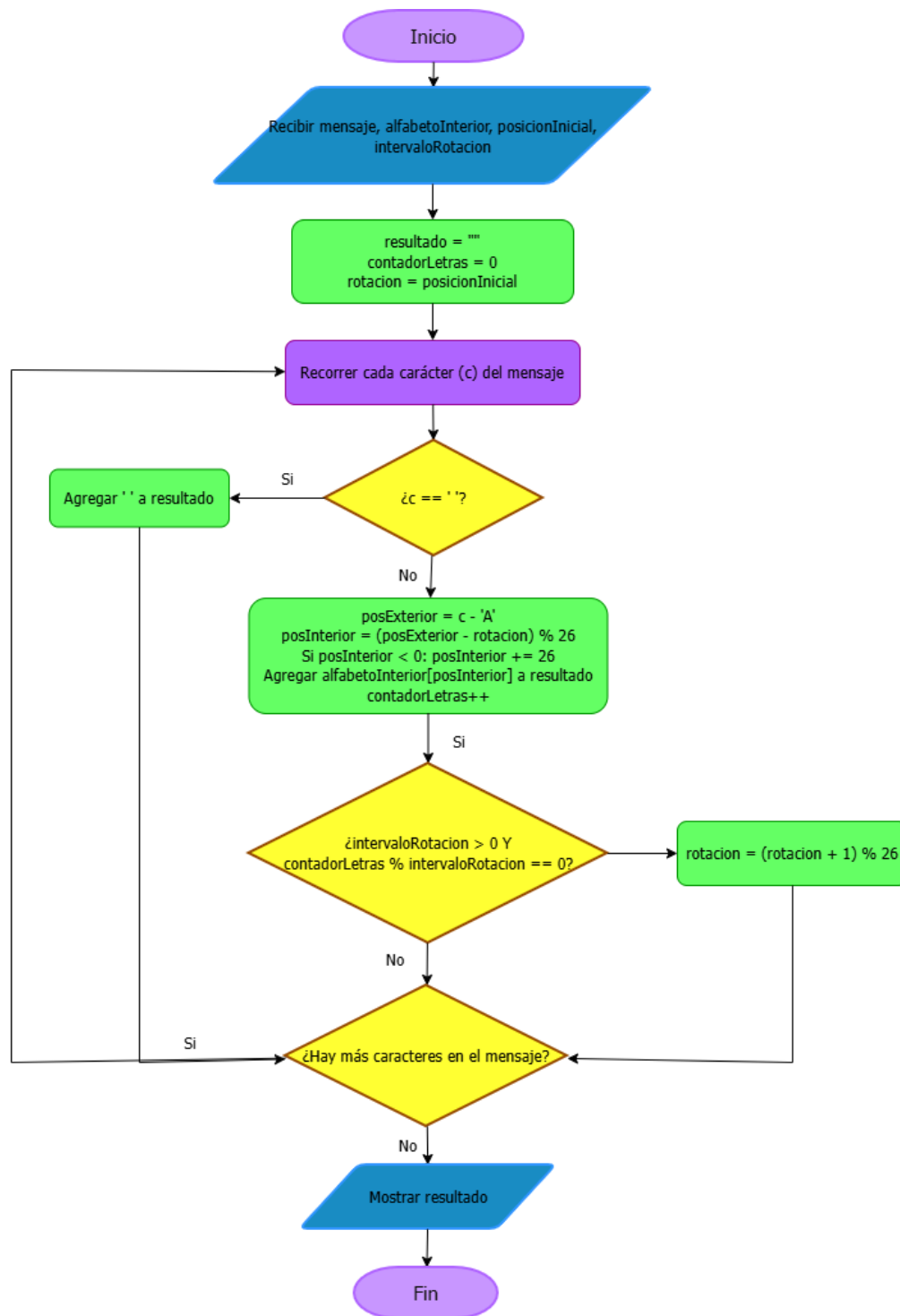
[Download](#)

✓ Test case 7 

✓ Test case 8 

✓ Test case 9 

Diagrama:



El programa analiza el mensaje carácter por carácter; si encuentra un espacio en blanco, lo conserva en el resultado sin alterar los contadores ni la posición de las ruedas.

Para las letras, calcula la posición correspondiente en la rueda interior aplicando un desplazamiento a la derecha según la rotación actual. Tras sustituir cada carácter, se

incrementa un contador de letras procesadas; al alcanzar un múltiplo del intervalo de rotación, la rueda interior gira una posición hacia la derecha para cambiar la clave de las letras siguientes. Al final, al terminar de recorrer la cadena, se devuelve el mensaje cifrado.

3. Pila dinámica

Una pila es una estructura de datos que sigue el principio LIFO (Last In, First Out): el último elemento en entrar es el primero en salir. En este ejercicio deberás construir una pila de valores double utilizando exclusivamente un arreglo.

La pila tendrá una capacidad inicial. Cuando el arreglo se encuentre lleno y se intente agregar un nuevo elemento, la operación no deberá rechazarse; se deberá crear un nuevo arreglo con el doble de capacidad, copiar los valores existentes, sustituir el arreglo anterior e insertar el nuevo elemento.

Actividad

Completa la clase PilaDinamica. El arreglo elementos será la única estructura utilizada para almacenar los valores y la variable cima deberá identificar la posición del último elemento agregado.

```
static class PilaDinamica {  
  
    private double[] elementos;  
    private int cima;  
  
    public PilaDinamica(int capacidadInicial)  
    public void push(double valor)  
    public double pop()  
    public double peek()  
    public boolean isEmpty()  
    public int size()  
    public int capacity()  
    private void ampliarCapacidad()  
  
}
```

Operaciones requeridas

Método	Comportamiento esperado
push(double valor)	Agrega el valor en la cima. Si el arreglo está lleno, primero duplica su capacidad.
pop()	Retira y devuelve el valor de la cima. Si está vacía, devuelve Double.NaN.

peek()	Devuelve el valor de la cima sin eliminarlo. Si está vacía, devuelve Double.NaN.
isEmpty()	Devuelve true cuando la pila no contiene elementos.
size()	Devuelve la cantidad de elementos actualmente almacenados.
capacity()	Devuelve la longitud actual del arreglo interno.
ampliarCapacidad()	Crea un arreglo con el doble de tamaño y copia manualmente los valores existentes.

Restricciones: No utilices Stack, ArrayList, LinkedList, Vector, Deque, Queue ni Arrays.copyOf() para implementar la pila. La copia de elementos deberá realizarse manualmente mediante un ciclo.

HackerRank ejecutará operaciones PUSH, POP, PEEK, SIZE, CAPACITY e ISEMPY para comprobar el comportamiento de tu clase. No necesitas implementar la lectura de estas operaciones; concéntrate únicamente en completar PilaDinamica.

Ejemplo de redimensionamiento:

```
Capacidad inicial = 3
[10.5][20.0][7.25]

PUSH 40.5

Capacidad nueva = 6
[10.5][20.0][7.25][40.5][ ][ ]
```

La operación POP sólo modifica la cima; no deberá reducir la capacidad del arreglo.

Evidencias: Prueba tu implementación mediante HackerRank, agrega una captura del código fuente y otra donde hayan pasado todos los casos de prueba. También recuerda subir tus códigos a Github.

Como parte del reporte, incluye una explicación y/o diagrama de flujo que demuestre claramente el razonamiento utilizado para resolver el problema, sin esto, no será válido el ejercicio.

Código fuente:

```
11  class Result {
13  static class PilaDinamica {
14
15      private double[] elementos;
16      private int cima;
17
18  public PilaDinamica(int capacidadInicial) {
19      this.elementos = new double [capacidadInicial];
20      this.cima = -1;
21  }
22
23  public void push(double valor) {
24      if (cima == elementos.length - 1){
25          ampliarCapacidad();
26      }
27      elementos [++cima] = valor;
28  }
29
30  public double pop() {
31      if (isEmpty()){
32          return Double.NaN;
33      }
34      return elementos [cima--];
35  }
36
37  public double peek() {
38      if (isEmpty()){
39          return Double.NaN;
40      }
41      return elementos [cima];
42  }
43
44  public boolean isEmpty() {
45      return cima == -1;
46  }
```

```

47
48 ✓      public int size() {
49         |         return cima + 1;
50         |     }
51
52 ✓      public int capacity() {
53         |         return elementos.length;
54         |     }
55
56 ✓      private void ampliarCapacidad() {
57         |         double[] nuevo = new double [elementos.length * 2];
58         |         for (int i = 0; i < elementos.length; i++){
59         |             nuevo [i] = elementos [i];
60         |         }
61         |         elementos = nuevo;
62         |     }
63     }

```

Ejecución:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

[Download](#)

```

1  3
2  5
3  CAPACITY
4  ISEMPY
5  SIZE
6  PEEK
7  POP

```

Your Output (stdout)

```

1  3
2  true

```

Congratulations

You solved this challenge. Would you like to challenge your friends?  

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

Compiler Message

Success

Input (stdin)

[Download](#)

```
1 3
2 5
3 CAPACITY
4 ISEMPY
5 SIZE
6 PEEK
7 POP
```

✓ Test case 7 

✓ Test case 8 

✓ Test case 9

Diagrama de flujo:



Durante la ejecución de *PUSH*, el algoritmo verifica si la posición de la cima ha alcanzado su límite de almacenamiento del arreglo actual; si se encuentra lleno, se desencadena *ampliarCapacidad()*, que crea un nuevo bloque de memoria con el doble de tamaño, que traslada secuencialmente cada valor existente con un ciclo manual y actualiza la referencia antes de posicionar el nuevo elemento e incrementar la cima.

Para *PEEK* y *POP*, primero evalúa si a la estructura le faltan elementos verificando que la cima sea igual a -1 . Si está vacía, retorna la constante *Double.NaN* (interpretada como *EMPTY*), evitando desbordamientos de memoria; de lo contrario, obtiene el valor presente en la posición actual de la cima y decrementa el puntero únicamente si se trata de *POP*, quedando intacta la capacidad del arreglo.

Preguntas de reflexión

1. ¿Por qué un arreglo en Java se considera un tipo de referencia? Explica qué ocurre con la referencia cuando se ejecuta `elementos = nuevo` después de ampliar la capacidad.

Un arreglo en Java es un objeto que se almacena en la memoria *Heap*. La variable del arreglo no contiene los datos directamente, sino una dirección de memoria (referencia) que apunta al lugar donde residen los elementos. Cuando se ejecuta `elementos = nuevo`, la variable `elementos` cambia su puntero de dirección para apuntar al nuevo arreglo de mayor tamaño alojado en el *Heap*. El arreglo anterior queda sin referencias activas que apunten a él, lo que permite que el Recolector de Basura (*Garbage Collector*) de Java libere automáticamente esa memoria previamente ocupada.

2. ¿Qué diferencia existe entre *String* y *StringBuilder* respecto a mutabilidad? ¿En qué situación del ejercicio de cifrado podría ser conveniente utilizar *StringBuilder*?

- **Mutabilidad:** Los objetos de tipo *String* son inmutables; cualquier modificación (como concatenar un carácter) genera un nuevo objeto *String* en memoria. Por otro lado, *StringBuilder* es mutable, permitiendo modificar, agregar o eliminar caracteres dentro de un mismo búfer dinámico sin instanciar objetos adicionales.
- **Aplicación en el cifrado:** En el ejercicio de la rueda del mensajero, se itera carácter por carácter sobre mensajes de hasta 10,000 letras. Utilizar *StringBuilder* evita crear miles de objetos *String* temporales en el *Heap*, optimizando drásticamente el tiempo de ejecución y el uso de memoria.

3. ¿Qué ventaja tiene duplicar la capacidad de un arreglo cuando se llena, en lugar de crear un arreglo sólo una posición más grande en cada *PUSH*? Considera tanto el número de copias como el uso de memoria.

- **Número de copias (Complejidad de tiempo):** Si el arreglo aumentará su tamaño en $+1$ en cada *PUSH*, se requeriría copiar todos los elementos existentes en cada inserción, resultando en un costo de $\sigma(N^2)$ para N elementos. Al duplicar la capacidad, la operación de copiado ocurre cada vez con menos frecuencia, logrando un costo de inserción amortizado $\sigma(1)$
- **Uso de memoria:** Aunque duplicar la capacidad genera un pequeño desperdicio temporal de memoria no utilizada (espacios vacíos), la reducción masiva de

operaciones de reasignación y copiado compensa ampliamente este gasto, garantizando un rendimiento óptimo en la estructura.

4. Una aplicación debe registrar miles de usuarios y comprobar rápidamente si un correo electrónico ya fue registrado antes de permitir una nueva alta. ¿Qué estructura utilizarías entre una lista, pila, cola, set o mapa para resolver este problema de manera eficiente? Justifica tu elección considerando la forma en que se realizan las búsquedas y la necesidad de evitar elementos duplicados.

La estructura ideal es un *Set*:

- **Búsqueda eficiente:** Los *Set* basados permiten realizar búsquedas e inserciones en tiempo constante $\sigma(1)$ promedio, a diferencia de una lista o cola que requieren una búsqueda lineal $\sigma(N)$.
 - **Unicidad:** La naturaleza de la interfaz *Set* prohíbe automáticamente la inserción de elementos duplicados, asegurando que cada correo sea único dentro del sistema sin necesidad de implementar validaciones manuales complejas.
5. ¿Qué ventajas y posibles riesgos tiene utilizar clases envolventes (*Integer*, *Double*, etc.) en lugar de tipos primitivos dentro de un programa? Relaciona tu respuesta con el uso de colecciones, valores *null* y conversiones automáticas mediante *autoboxing* y *unboxing*.

Ventajas:

- **Uso en colecciones:** Las clases envolventes (*Wrappers*) permiten trabajar con el marco de colecciones de Java (como *ArrayList<Double>*), el cual solo acepta objetos y no tipos primitivos.
- **Representación de datos:** Admiten el valor *null*, lo que resulta indispensable para representar ausencia de valor o estados no inicializados (por ejemplo, en bases de datos).

Riesgos:

- **Riesgo de *NullPointerException*:** Al realizar *unboxing* automático de un objeto envolvente que contiene un valor *null* a un tipo primitivo, el programa lanzará una excepción en tiempo de ejecución.
- **Sobrecostos de memoria y rendimiento:** Las clases envolventes consumen significativamente más memoria en el *Heap* que un tipo primitivo y añaden tiempo de procesamiento extra debido a las conversiones continuas entre primitivos y objetos (*autoboxing* y *unboxing*).

Conclusión:

En la realización de esta práctica se analizaron y aplicaron conceptos fundamentales de estructuras de datos lineales y algoritmos de manipulación de arreglos en Java. La resolución del problema de la montaña rusa permitió poner en práctica estrategias de búsqueda mediante punteros para delimitar y optimizar la reordenación de subarreglos en tiempo lineal. Por otro lado, la implementación del cifrado de la rueda del mensajero evidenció la importancia de la aritmética modular y la manipulación de cadenas mediante estructuras mutables como `StringBuilder` para garantizar un procesamiento eficiente en memoria. Finalmente, el desarrollo de la clase `PilaDinamica` reafirmó el principio LIFO y demostró cómo un arreglo estático puede redimensionarse dinámicamente para lograr un costo amortizado constante, reforzando la comprensión sobre el manejo de memoria en el *Heap* y el uso de referencias en Java.

Bibliografía:

- Cairó, O., y Guardati, S. (2006). *Estructuras de datos* (3.^a ed.). McGraw-Hill / Interamericana de España. <https://www.mheducation.es>
- Joyanes Aguilar, L., y Zahonero Martínez, I. (2008). *Estructuras de datos en Java* (1.^a ed.). McGraw-Hill. <https://www.mheducation.es>
- Universidad Nacional Autónoma de México. (2020). *Estructuras de Datos y Algoritmos: Notas de clase*. Facultad de Ingeniería, UNAM. <https://www.ingenieria.unam.mx>